

RUHR-UNIVERSITÄT BOCHUM

Fundamentals of GPU Programming
Lecture 1

Denis Eremin

Fakultät für Elektrotechnik und Informationstechnik
Lehrstuhl für Theoretische Elektrotechnik

Outline

- Grading Policy
- Lecture materials
- Evolution of computational resources
- General purpose GPU computing
- CUDA Programming Model

Check announcements on the corresponding site of the moodle course for this class!

Grading Policy

- Homeworks (submitted via Moodle) (50%)
  Work in groups ok, using AI (any) **NOT OK**

- Written Exam (50%)

Info Sources

- NVIDIA CUDA C Programming Guide
  (https://docs.nvidia.com/cuda/cuda-programming-guide/)
- NVIDIA CUDA C Best Practices Guide
  (http://docs.nvidia.com/cuda/cuda-c-best-practices-guide/)
- Programming Massively Parallel Processors: A Hands-On Approach
  (Hwu, Kirk, Hajj) 5th Ed.
- Lecture notes
  (uploaded to the moodle)

NVIDIA Webinars

https://www.nvidia.com/object/webinar-portal.html

NVIDIA CUDA Developer Forums

https://forums.developer.nvidia.com/

Computer Hardware Evolution

Single core CPUs (... - 2003)

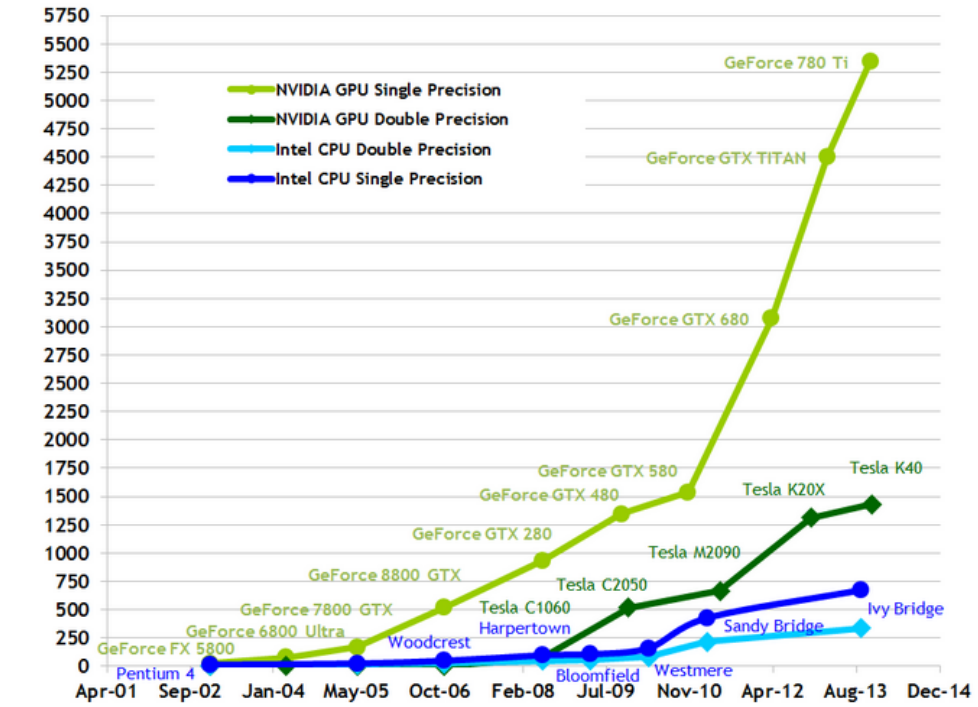
CPU-based clusters (1995-...)

Multi-core CPUs (2003 - ...)

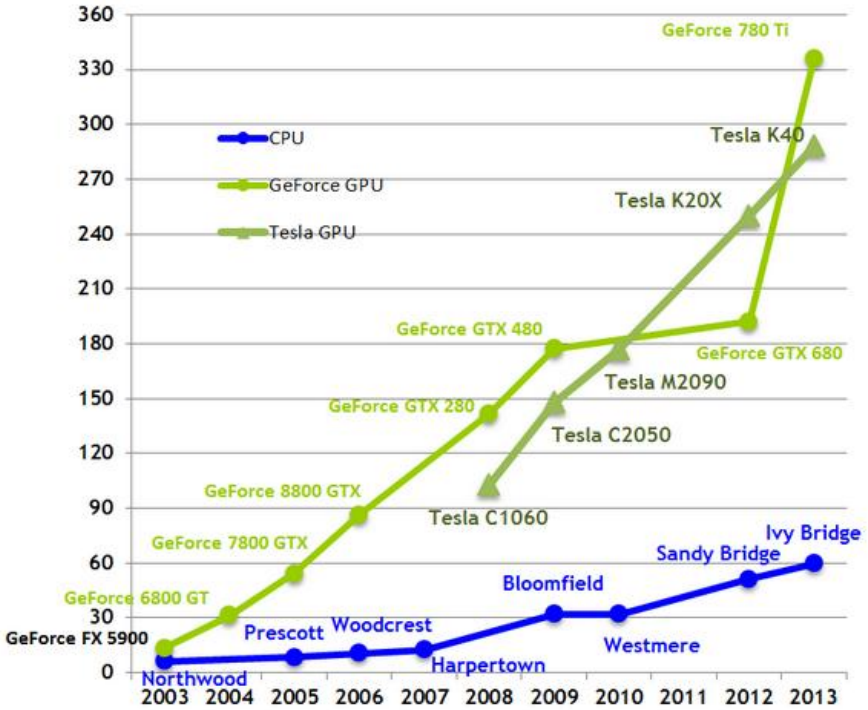
Many-core (many-thread) GPUs (2003 - ...)

GPU Performance

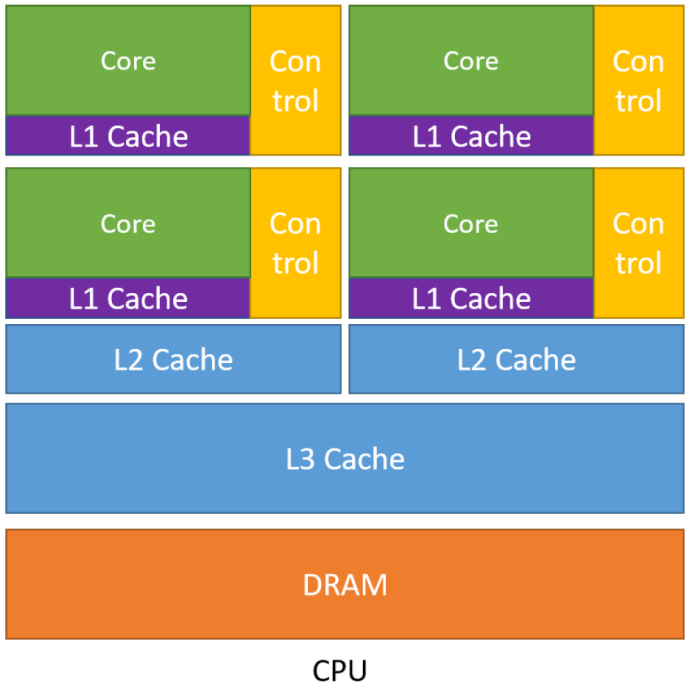
Theoretical GFLOP/s



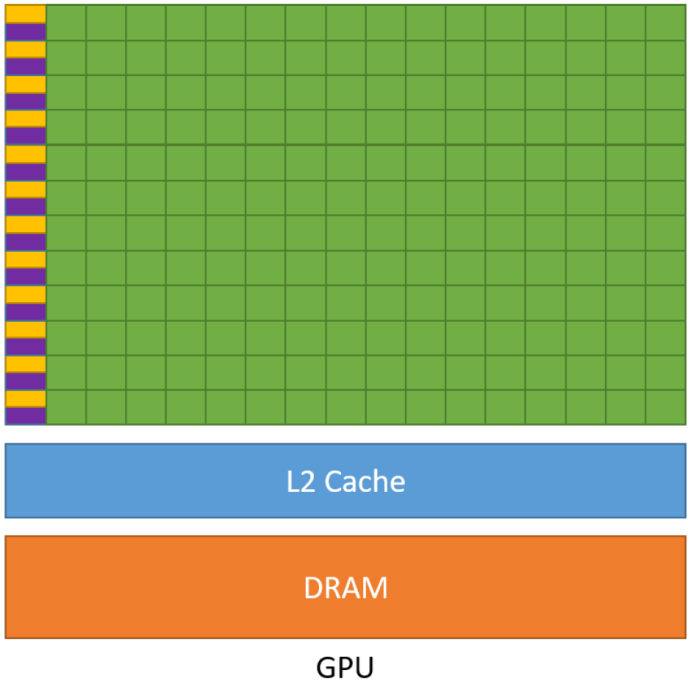
Theoretical GB/s



Multi-core CPUs vs. Many-core GPUs



CPU



GPU

Latency-oriented design

- MIMD: sophisticated flow control
- Large overhead to launch a new thread
- Large low-latency cache is needed to speed up memory access

Throughput-oriented design

- SIMD: lower requirement for flow control
- Large number of light threads (low overhead)
- Massively parallel calculations, hence
  - high arithmetic intensity
  - memory latency can be hidden by switching between threads, no need for a large cache

Problem Types

Parallelizable Problem

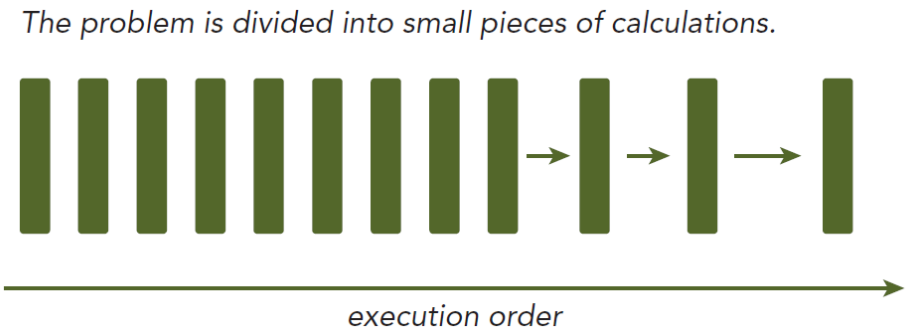
Vector addition: $c(i) = a(i) + b(i)$, $i = 0..N-1$

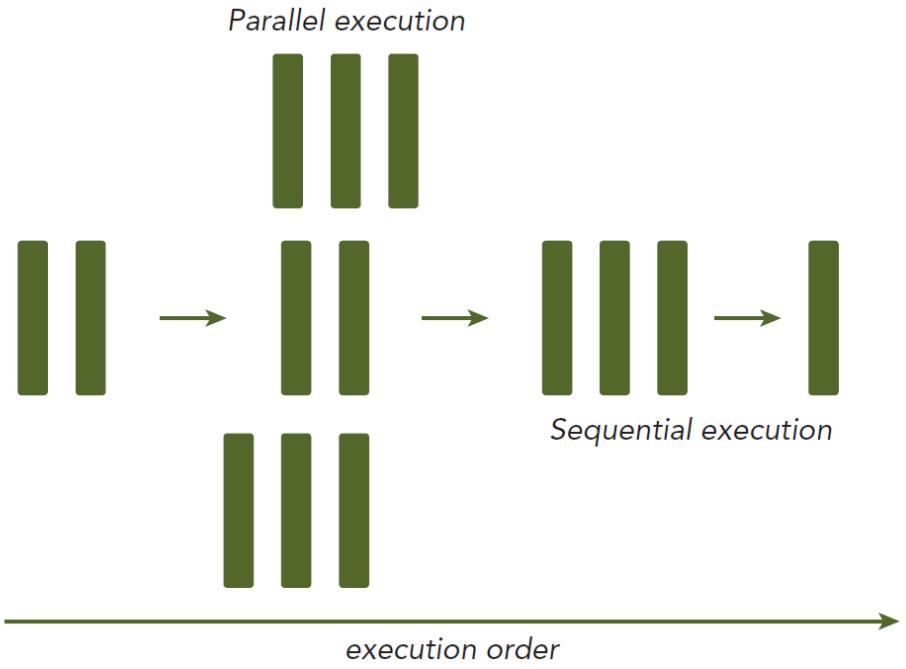
Non-parallelizable Problem

Calculate of the Fibonacci series (0,1,1,2,3,5,8,13,21,...) by use of the formula:

$$F(n) = F(n-1) + F(n-2)$$

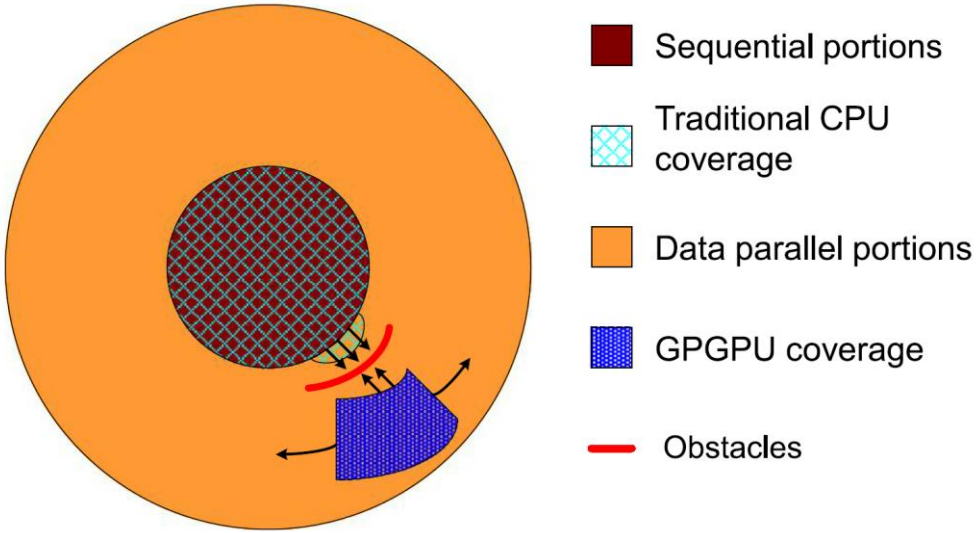
Real-life Problems





Amdahl's law





$$\text{Speedup} = \frac{1}{(1-p)+p/N}$$

Attractivity of GPUs as Commercial HPC Means

- Very dynamic evolution (driven by the Computer Game Industry)
- High performance
- Up to x100 speedup compared to CPU in certain problems (real-time analysis becomes possible)
- Low cost
- Abundance of programming and development tools
- CUDA C is a straightforward extension of C (low learning curve)
- Accessible form-factor
- Large market share

Example: a Gamer GPU (RTX 3080)

Graphics Processor	
GPU Name:	GA102
GPU Variant:	GA102-200-KD-A1
Architecture:	Ampere
Foundry:	Samsung
Process Size:	8 nm
Transistors:	28,300 million
Die Size:	628 mm²

Clock Speeds	
Base Clock:	1440 MHz
Boost Clock:	1710 MHz
Memory Clock:	1188 MHz 19 Gbps effective

Board Design	
Slot Width:	Dual-slot
Length:	285 mm 11.2 inches
Width:	112 mm 4.4 inches
Height:	40 mm 1.6 inches

Graphics Card	
Release Date:	Sep 1st, 2020
Availability:	Sep 17th, 2020
Generation:	GeForce 30
Predecessor:	GeForce 20
Successor:	GeForce 40
Production:	Active
Launch Price:	699 USD
Current Price:	Amazon / Newegg
Bus Interface:	PCIe 4.0 x16
Reviews:	103 in our database

Theoretical Performance	
Pixel Rate:	164.2 GPixel/s
Texture Rate:	465.1 GTexel/s
FP16 (half) performance:	29.77 TFLOPS (1:1)
FP32 (float) performance:	29.77 TFLOPS
FP64 (double) performance:	465.1 GFLOPS (1:64)

Relative Performance

GeForce RTX 3070	77%
GeForce RTX 3070 Ti	82%
Radeon RX 6800	85%
Radeon RX 6800 XT	97%
GeForce RTX 3080	100%
Radeon RX 6900 XT	106%
GeForce RTX 3080 Ti	112%
Radeon RX 6950 XT	112%
GeForce RTX 3090	113%
GeForce RTX 3090 Ti	122%
GeForce RTX 4090	172%

Based on TPU review data: "Performance Summary" at 1920x1080, 4K for 2080 Ti and faster.

Memory	
Memory Size:	10 GB
Memory Type:	GDDR6X
Memory Bus:	320 bit
Bandwidth:	760.3 GB/s

Graphics Features	
DirectX:	12 Ultimate (12_2)
OpenGL:	4.6
OpenCL:	3.0

Render Config	
Shading Units:	8704
TMUs:	272
ROPs:	96
SM Count:	68
Tensor Cores:	272
RT Cores:	68
L1 Cache:	128 KB (per SM)
L2 Cache:	5 MB

CUDA vs GPGPU

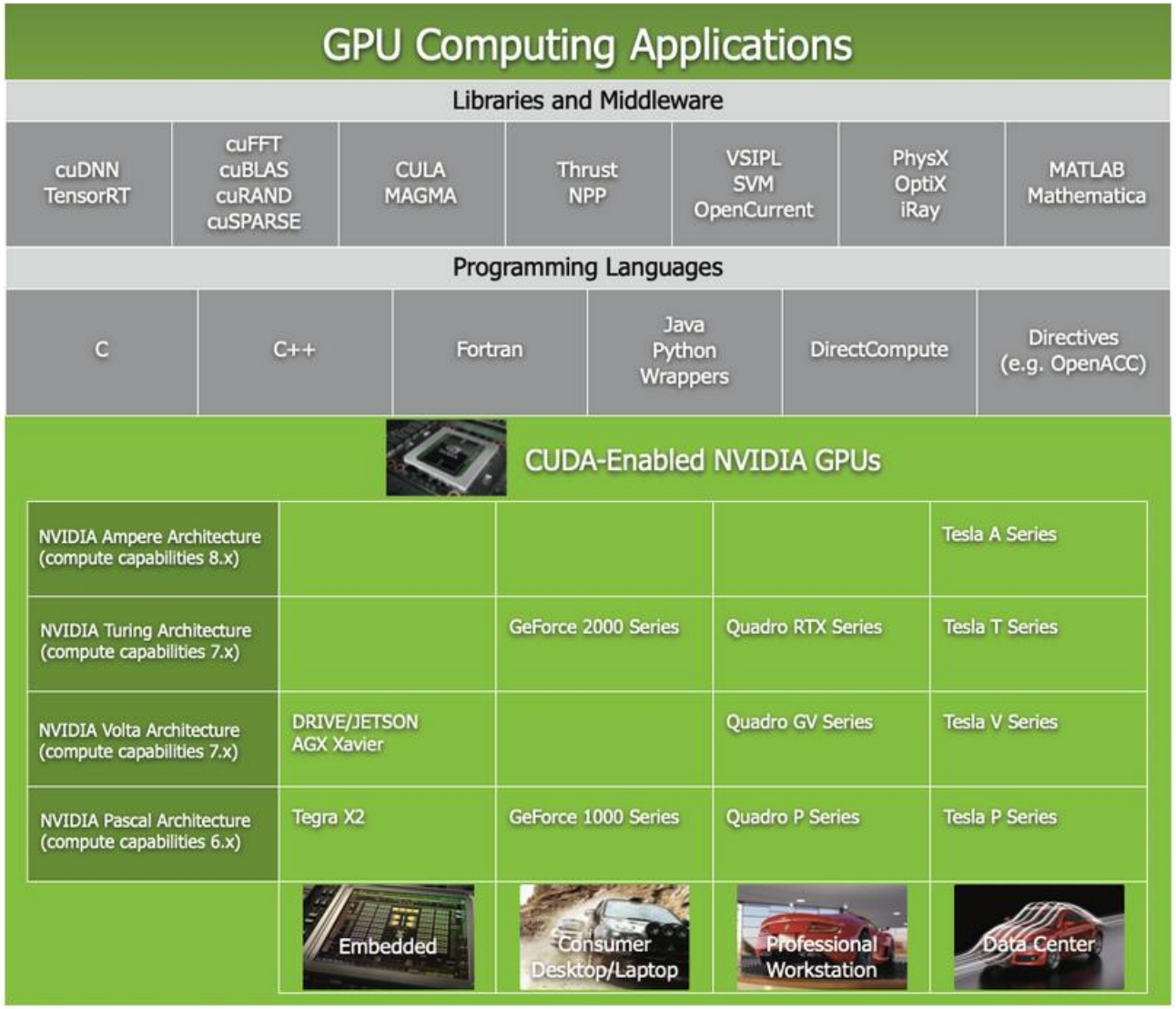
GPGPU (2003-2006)

- Use of GPU for general-purpose computations. GPGPU meant programming in the complex and unnatural language of GPU (e.g., OpenGL)
- Memory access problems, performance overhead of graphics layer APIs.
- Lack of integer and bitwise operations

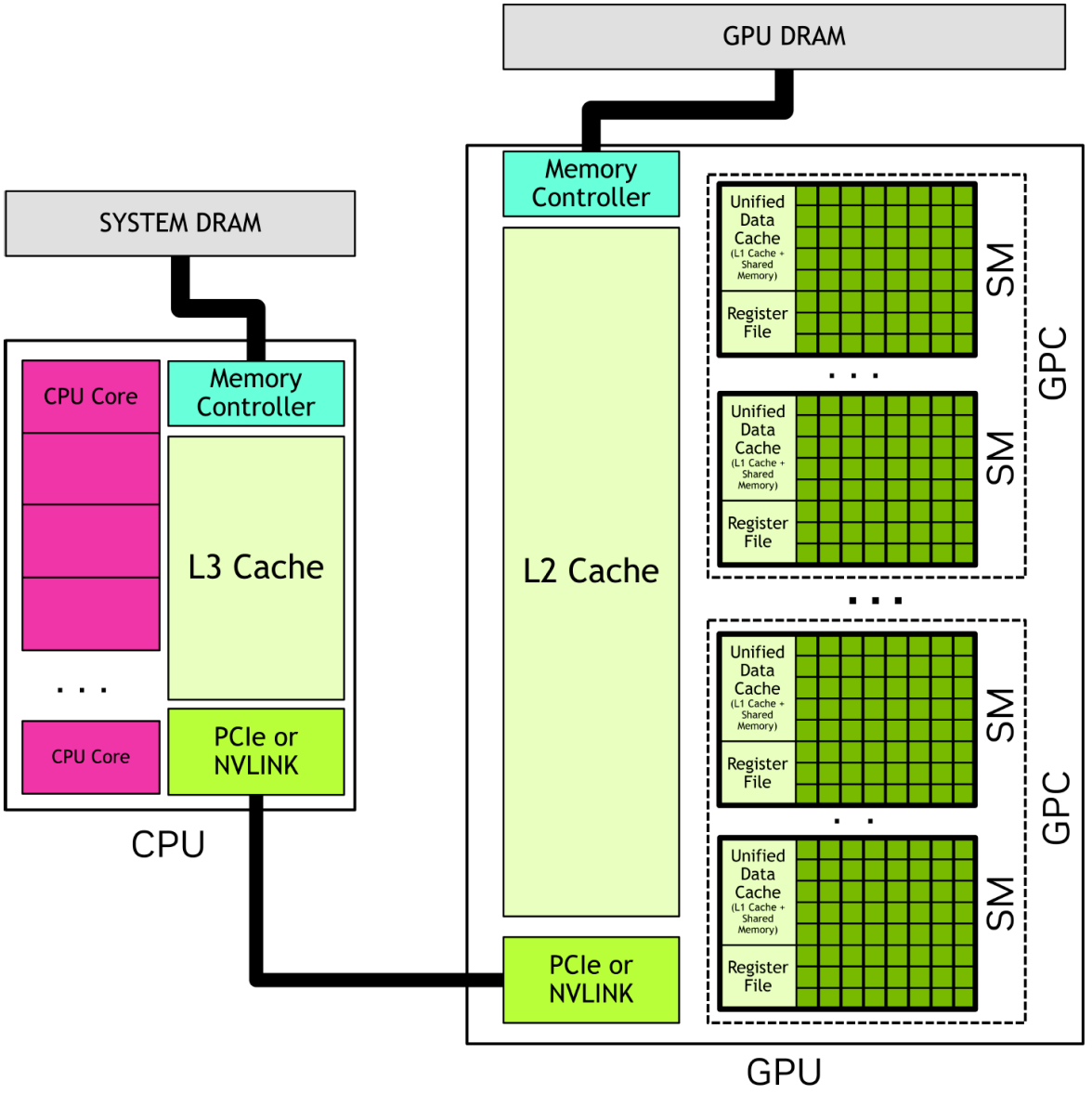
CUDA (2007-Present)

- CUDA = (**C**ompute **U**nified **D**evice **A**rchitecture)
- CUDA permits working with familiar programming concepts while developing software that can run on a GPU.
- Avoids the performance overhead of graphics layer APIs by compiling your software directly to the hardware
- Explicit GPU memory management

Other High-Level Languages for GPU Programming



Heterogeneous CPU-GPU System



CUDA Parallel Programming Model

Challenge: to guarantee scalability with the number of GPU cores

Solution: three levels of abstraction of the CUDA Programming Model

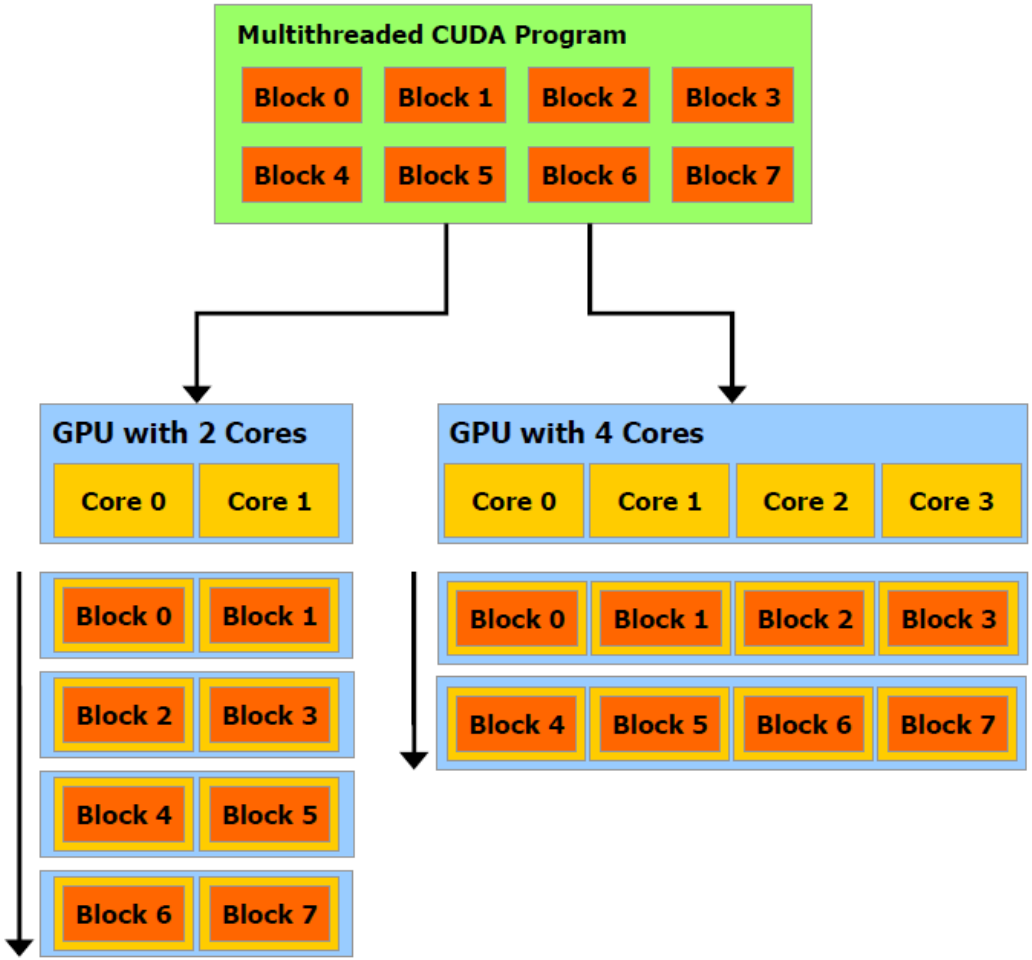
- Hierarchy of thread groups
- Hierarchy of memory types
- Barrier synchronization

Problem decomposition:

Problem → Coarse – grained parallelism (independent thread blocks) → Fine – grained parallelism (threads within blocks)

Thread blocks can be scheduled for execution on the hardware in any order

CUDA Scalable Programming Model



Kernels

Kernel is a function to be executed on Device

kernel function specifier

```
// Kernel definition
__global__ void VecAdd(float* A, float* B, float* C)
{
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}

int main()
{
    ...
    // Kernel invocation with N threads
    VecAdd<<<1, N>>>(A, B, C);
}
```

kernel declaration

kernel launch

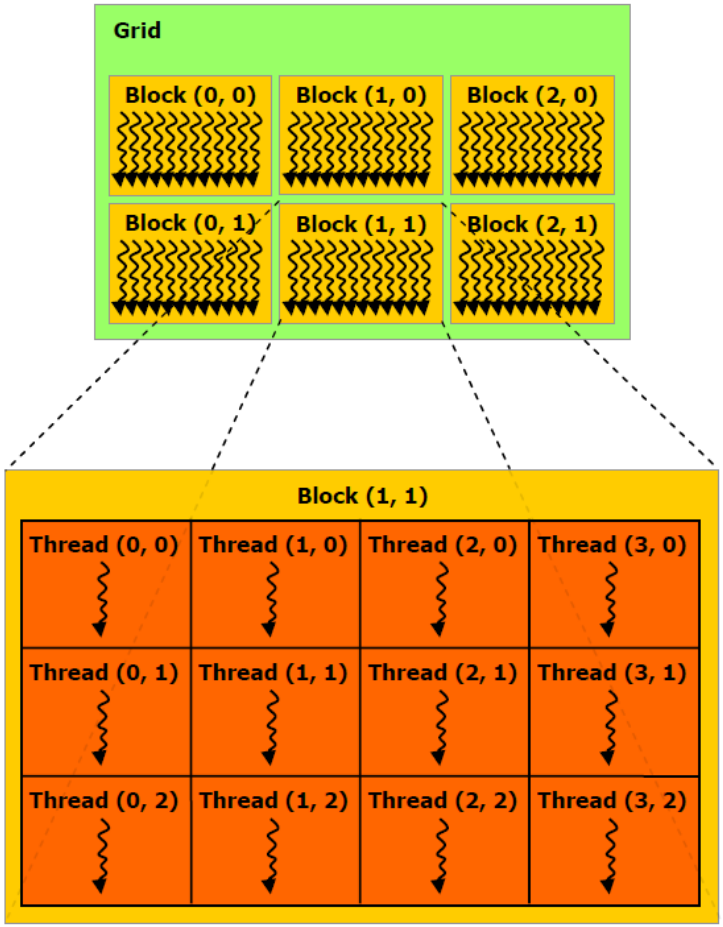
execution configuration

Each thread is accessible through a unique $0 \leq \mathrm{threadIdx} \leq N - 1$

$N \leq 1024$ (kernel does not launch if violated)

Thread Coordinates (Built-in Variables)

[threadIdx (max. blockDim) , blockIdx (max. gridDim)]

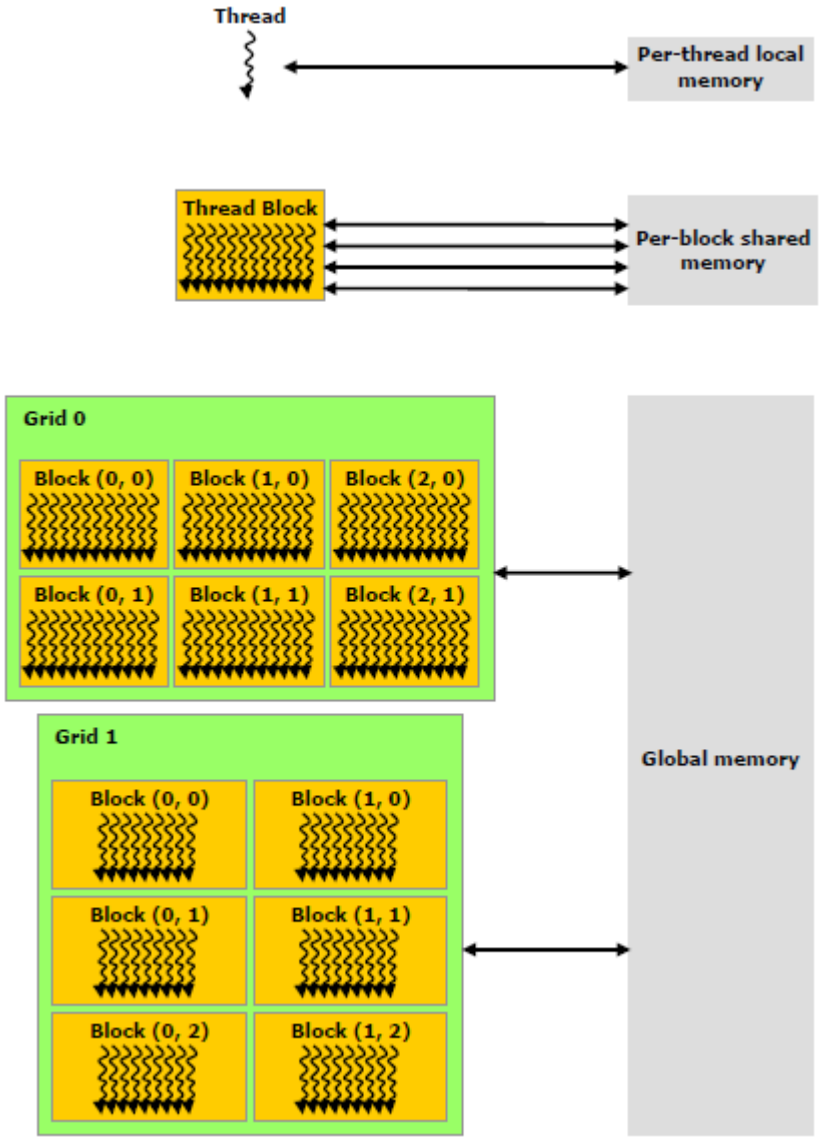



```
// Kernel definition
__global__ void MatAdd(float A[N][N], float B[N][N],
                       float C[N][N])
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    if (i < N && j < N)
        C[i][i] = A[i][i] + B[i][i];
}

int main()
{
    ...
    // Kernel invocation
    dim3 threadsPerBlock(16, 16);
    dim3 numBlocks(N / threadsPerBlock.x, N / threadsPerBlock.y);
    MatAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
}
```

Number of blocks is defined by the data size to be processed or number of processors (to make the GPU load uniform)

Device Memory Hierarchy

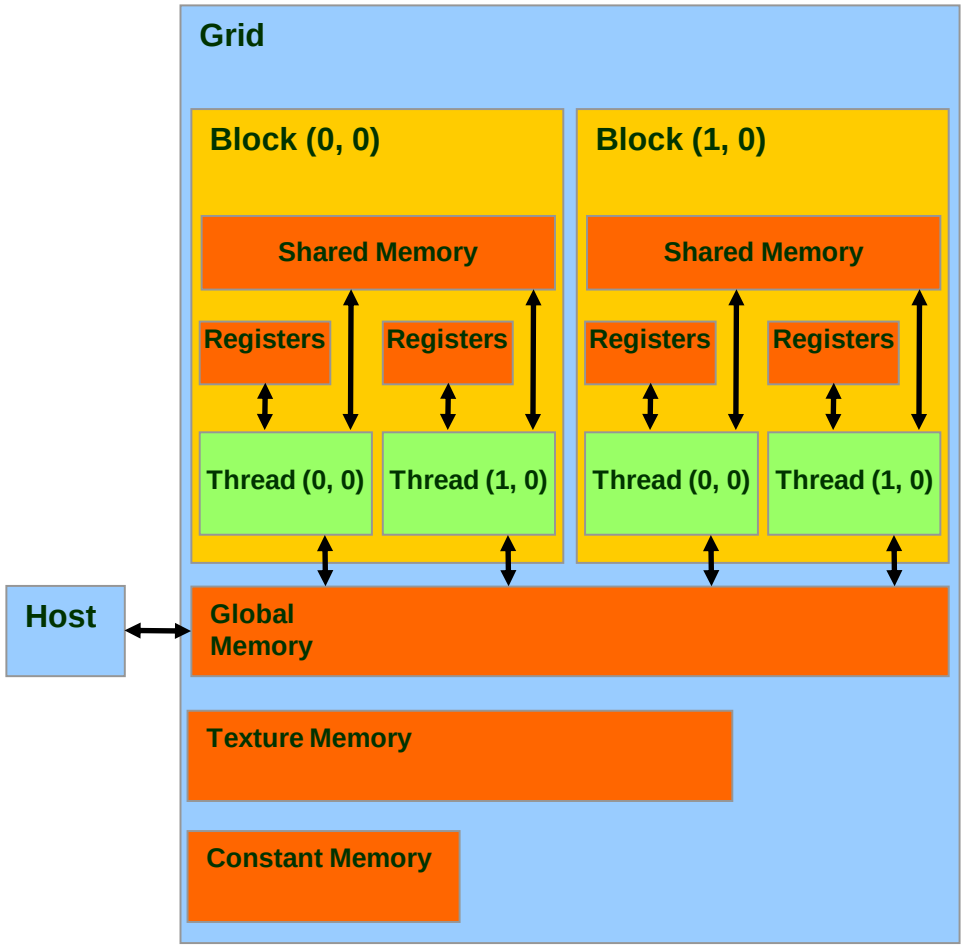


register and local memory
(lifetime of a thread)

shared memory
(lifetime of a block)

global, texture, constant memory
(lifetime of an application)

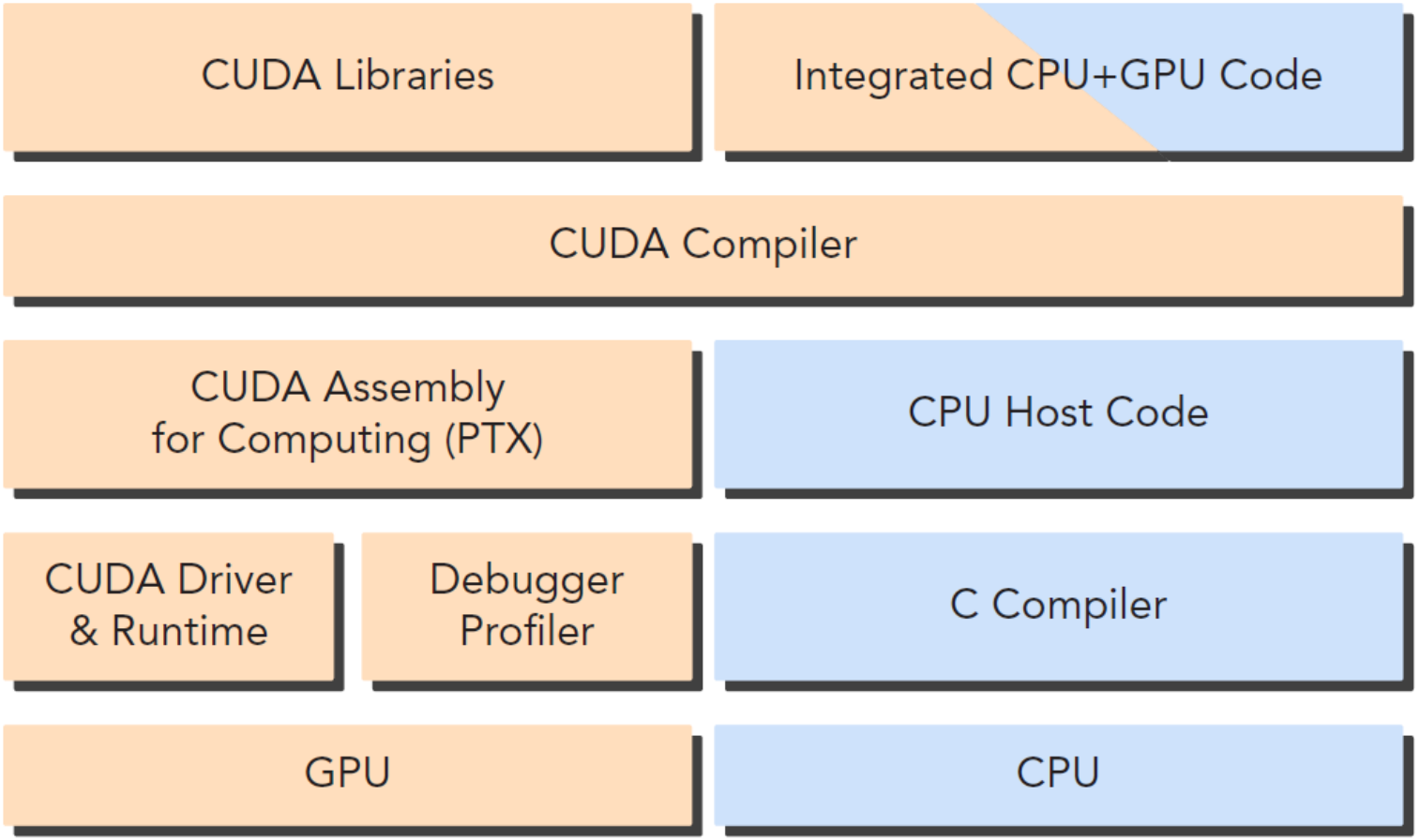
Device Memory Hierarchy (Contd.)



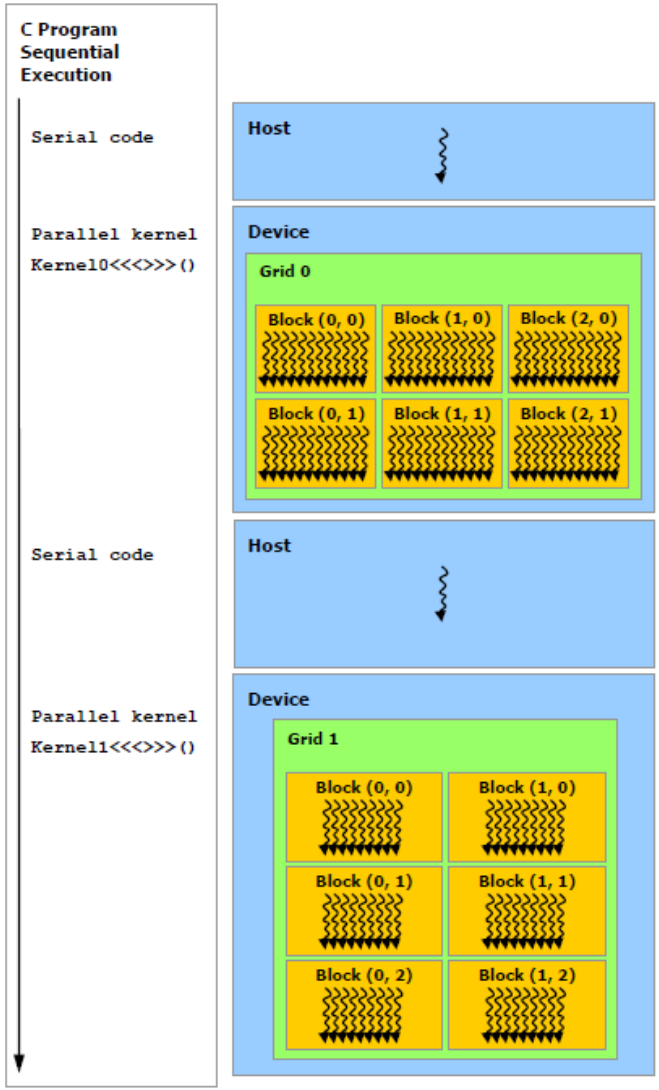
Barrier Synchronization Hierarchy

`__syncthreads()`,	Threads in a block must wait until the barrier is reached by all of them
`cudaDeviceSychronize()`	CPU waits until a GPU task is done

Heterogeneous CPU/GPU Code Compilation



Heterogeneous CPU/GPU Code Execution



Kernel = Function executed on Device

Grid = All threads generated by Kernel

Note: execution on GPU is asynchronous, i.e. independent of CPU unless a barrier is invoked!

Homework#1 (not graded): Hello World

Install CUDA environment. Compile with nvcc, and run helloWorld.cu :

```
#include <stdio.h>

// hello from GPU kernel
__global__ void helloFromGPU(void)
{
  printf("Hello World from GPU! \n");
}


int main(void)
{
   // hello from gpu
   helloFromGPU <<<1, 10>>>();
   cudaDeviceReset();
   return 0;
}
```

CUDA Download and Installation

https://developer.nvidia.com/cuda/cuda-downloads

- Developer drivers
- CUDA Toolkit
- SDK code samples

Compilation and Linking

- Any source file containing CUDA language extensions must be compiled with NVCC

- NVCC is a compiler driver
  - Works by invoking all the necessary tools and compilers like cudacc, g++, cl, ...

- Any executable with CUDA code requires two dynamic libraries when linking:
  - The CUDA runtime library (**`cudart`**)
  - The CUDA core library (**`cuda`**)

Profiling/Debugging

Visual tools:

Nsight Compute/Systems

Command-line tools:

nprof, compute-sanitizer

If you don’t have an NVIDIA GPU:

Students from ETIT: check working hours of you CIP-Insel, you should have access to computers with such GPUs there. There should be admins that help you. In case you’re totally overwhelmed, contact Thomas Pflanz thomas.pflanz@ruhr-uni-bochum.de

Students from other faculties: contact our sysadmin Ke Yan Ke.Yan@tet.ruhr-uni-bochum.de